

Guion de defensa — 15 minutos

Fine-Tuning eficiente (LoRA/QLoRA) de LLMs pequeños para análisis de sentimiento en español

Material de preparación — Fase 3 (para decir en voz alta)

Cómo ensayar. Cada bloque lleva su *tiempo acumulado* (esquina derecha) y un *mensaje clave* de una frase: si te pierdes, vuelve a esa frase. Cada guión (–) es una frase corta, pensada para decirse de un respiro. Las marcas □ [apoyo visual] son sugerencias; las ▷ **Pre-empción** son los puntos donde *tú* sacas la debilidad antes que el tribunal. Ritmo objetivo: ≈ 130 – 140 palabras/min. Presupuesto: **13:30–14:00** hablado (contenido $\approx 13:45$, *incluida la demo de cierre en vivo*). El resto del cuarto de hora es margen para respiraciones, una aclaración a media charla y el alargue natural del directo.

1. Problema

[00:00 → 01:45 · 1 min 45 s]

Mensaje clave: *Adaptar un modelo de lenguaje a una tarea era caro; la pregunta de este trabajo es cuándo y a qué coste deja de serlo.*

□ [apoyo visual]: Título + tres tuits de ejemplo etiquetados +/neutral/–.

- Buenos días. Os hablo de cómo adaptar modelos de lenguaje al análisis de sentimiento en español... sin pagar una GPU cara.
- El problema es fácil de enunciar: hay millones de tuits en español con opiniones, y queremos clasificarlos en positivo, negativo o neutral.
- Pero lo difícil no es solo acertar. Es acertar *barato*.
- Adaptar un modelo grande a la antigua —reescribiendo todos sus pesos— exige hardware caro y muchos datos etiquetados. Para casi todo el mundo, eso es inviable.
- Han aparecido técnicas que prometen hacerlo con una fracción del coste: LoRA y QLoRA. Pero para el español quedan preguntas sin responder.
- Y de esas preguntas va este trabajo: ¿cuándo merece la pena afinar?, ¿a qué coste?, ¿y sirve para distintas variedades del español?

2. Método / Propuesta

[01:45 → 04:00 · 2 min 15 s]

Mensaje clave: *Un estudio empírico reproducible en hardware gratuito: LoRA/QLoRA sobre dos Qwen3 pequeños, frente a los encoders de referencia, en tres cortes.*

□ [apoyo visual]: Diagrama: modelos (Qwen3 1.7B/4B) · datasets (Cardiff, InterTASS) · los 3 cortes A/B/C.

- Mi enfoque es un estudio empírico y reproducible. Afino dos modelos abiertos y pequeños: Qwen3 de 1.700 y de 4.000 millones de parámetros, que caben en una GPU gratuita de 16 GB.
- Como referencia, los encoders clásicos del español —BETO y XLM-RoBERTa— afinados a la antigua.
- Los datos: tuits de Cardiff para las dos primeras preguntas; e InterTASS —España, Costa Rica y Perú— para la tercera.

- Mido con macro-F1: la media de la F1 de las tres clases. Las trata por igual, así que no se puede ignorar la clase difícil, el *neutral*.
- Y lo organizo en tres cortes: A, ¿cuándo afinar gana al prompting?; B, ¿gana LoRA a los encoders contando el coste?; C, ¿un adaptador de una variedad sirve para otra?
- Dos garantías de rigor: los hiperparámetros de LoRA están *fijos* en todos los experimentos, y el test se toca una sola vez, al final.

3. Resultados

[04:00 → 08:45 · 4 min 45 s]

3a. Corte A — punto de cruce 04:00 → 05:15 **Mensaje clave:** *El punto en que afinar supera al prompting no es universal: depende del tamaño del modelo.*

□ [apoyo visual]: Figura A: macro-F1 vs n (escala log), con el cruce marcado para 1.7B y 4B.

- Primer resultado, el corte A.
- Comparo afinar con LoRA frente a solo “pedirle” la respuesta al modelo con ejemplos en el prompt.
- Para el modelo de 1.7B, afinar gana enseguida: con menos de **siete** ejemplos etiquetados ya supera a su mejor prompting.
- Pero con el de 4B la historia cambia. Como de fábrica ya lo hace mejor, necesita más datos: no cruza con firmeza hasta unos **cincuenta** ejemplos.
- La conclusión no es un número mágico. Es un mecanismo: cuanto mejor es el modelo sin afinar, más datos necesita el fine-tuning para superarlo.

Cifras: Tabla II — 1.7B mejor prompting 0.560, LoRA 0.590 en $n=7$; 4B mejor prompting 0.652, LoRA 0.666 en $n=50$.

3b. Corte B — frontera de Pareto 05:15 → 08:00 **Mensaje clave:** *LoRA domina la frontera calidad-coste: más calidad que los encoders entrenando una fracción de los parámetros.*

□ [apoyo visual]: Figura B: Pareto calidad vs parámetros (log). LoRA arriba a la izquierda; RoBERTuito marcado fuera de la frontera.

- Segundo resultado, el corte B, y el más visual.
- A datos completos, LoRA gana a los encoders. El 4B llega a **0,707** de macro-F1; el 1.7B, a **0,698**. BETO se queda en 0,661; XLM-RoBERTa, en 0,646.
- Y lo hace entrenando una fracción mínima: frente a los encoders, entre **17 y 43 veces** menos parámetros con el 1.7B, y entre **9 y 24 veces** menos con el 4B.
- Comparado con afinar el modelo entero, son **268 veces** menos parámetros. . . con prácticamente la misma calidad. Afinar el modelo entero no compensa.

▷ **Pre-empción W1 — RoBERTuito (una frase y avanzar)**

- Un apunte honesto: en la tabla hay un modelo de fábrica, RoBERTuito, con 0,758; lo reporto, pero queda *fuera* de la frontera a propósito, porque está afinado en otro corpus y lo evalué sin reentrenar —no es comparable en igualdad de condiciones.
- Contra mis baselines controlados —BETO y XLM-R, mismo protocolo— LoRA gana. Y sigo.
- Y una pieza más: QLoRA. Cuantizando a 4 bits, retiene entre el **97 y el 99 por ciento** de la calidad usando aproximadamente la mitad de memoria.
- Eso mete el modelo de 4B en una tarjeta de consumo de 8 GB.

Cifras: Tabla IV — LoRA 4B 0.707 / 1.7B 0.698; BETO 0.661, XLM-R 0.646; 17-43× (1.7B), 9-24× (4B), 268× vs full-FT; QLoRA 4B 0.701 a 6.79 GB (vs 16.7), 1.7B 0.681 a 4.93 GB (vs 8.72).

3c. Corte C — transferencia dialectal 08:00 → 08:45 **Mensaje clave:** *Un adaptador transfiere casi siempre sin coste entre variedades; la excepción, replicada, es apuntar al español de España.*

□ [apoyo visual]: Figura C: los dos heatmaps 3×3 (1.7B y 4B), diagonal resaltada. (Detalle 3×3 → explorador, Vista 3, en Q&A.)

- Tercer resultado, el corte C, en un titular: un adaptador entrenado en una variedad del español sirve para otra casi sin coste.
- Entreno en una y evalúo en otra —España, Costa Rica, Perú—: hacia Costa Rica y Perú es gratis; la única excepción es apuntar al español de España, con una penalización de entre **0,031 y 0,039** de F1.
- Y lo que le da peso no es el número, sino que la asimetría se *repite* en los dos tamaños de modelo. El detalle 3×3 lo tengo en el explorador, si preguntáis.

Cifras: Tabla III — penalización a ES 0.031–0.039 (2.6–4.6σ), replicada en 1.7B y 4B; transferencia a CR/PE dentro del ruido.

4. Límites honestos

[08:45 → 11:15 · 2 min 30 s]

Mensaje clave: *Los límites están declarados de antemano; varios resultados “negativos” no son fallos, son la prueba de que el experimento medía lo que decía.*

□ [apoyo visual]: Lista breve: dev-split · sin test formal · H200 vs T4 · un solo dataset.

- Paso a los límites, porque la honestidad es parte del trabajo.
- Sobre la penalización del corte C: en la memoria soy explícito. La describo en múltiplos del ruido entre semillas —hasta 4,6 veces— pero eso es una medida de *magnitud*, no un test estadístico. No vendo una significancia que no he calculado.
- El corte C, además, lo evalúo en el conjunto de desarrollo, no en el de test. ¿Por qué? Porque el test oficial estaba inaccesible: daba error 404.
- El de desarrollo nunca se usó para entrenar ni para seleccionar, así que no hay trampa; solo aviso de que no es comparable con resultados ajenos sobre el test.

▷ Pre-empción W4 — H200 vs T4 (una frase y avanzar)

- Una aclaración sobre el hardware: el título habla de una T4 gratuita de 16 GB, pero corrí en una H200.
- No es contradicción: lo que sostiene la accesibilidad es la *memoria pico*, entre **4,9 y 8,7 GB** —las cifras del objetivo T4— y eso cabe en 16 GB corra donde corra.
- La H200 solo paralelizó el barrido; lo único que cambiaría en una T4 es el tiempo. Y avanzo.
- Hay más límites declarados: un solo dataset en los cortes A y B, corpus pequeños en el C, y dominio Twitter. Ninguno invalida los hallazgos centrales.
- Y un apunte, del lado bueno: probé la robustez a ruido real —tildes, emojis, abreviaturas, mayúsculas— y el único colapso claro es de los encoders *cased* ante el mayúsculado (BETO pierde **0,18** de macro-F1; los LoRA, 0,01). Es un límite de los *baselines*, no del método —y os lo enseño en vivo al cerrar.

Cifras: §V y §VI — VRAM pico 4.93–8.72 GB (Tabla IV); test InterTASS HTTP 404; $2.6\text{--}4.6\sigma$ = heurístico, no test formal. Robustez: mayúsculas BETO -0.184 / XLM-R -0.110 (`results/robustness_results.csv`, Fig. F).

5. Contribución

[11:15 → 12:15 · 1 min]

Mensaje clave: *La aportación no es un récord de F1, sino un mapa reproducible de cuándo, cuánto y a qué coste conviene cada opción.*

□ [apoyo visual]: Tres contribuciones, una línea cada una; abajo, la palabra “reproducible”.

- ¿Qué apporto? Tres cosas y un hilo común.
- Primero, el punto de cruce prompting-a-fine-tuning para el español: depende del tamaño del modelo.
- Segundo, un benchmark calidad-coste controlado donde LoRA domina a los encoders en hardware gratuito.
- Tercero, una primera evidencia de asimetría dialectal en la transferencia, replicada en dos modelos.
- Y el hilo común: reproducibilidad —código, configuraciones y particiones liberadas—. No reclamo el mejor F1 absoluto, sino un mapa útil para quien decide con poco presupuesto.

6. Cierre en vivo — demo

[12:15 → 13:45 · 1 min 30 s]

Mensaje clave: *No es una tabla más: el adaptador clasifica en directo, y el colapso cased se ve en vivo —los encoders voltean con el mayúsculado, los LoRA aguantan.*

□ [apoyo visual]: Demo en vivo servida desde la DGX (H200) por túnel SSH: los 5 modelos a la vez. Slide de respaldo con el antes/después estático por si falla (Plan B).

□ **Preparación (antes de la charla):** demo ya arrancada y con warmup hecho en la DGX, túnel SSH abierto y probado, navegador en `localhost:7860`, frase lista para teclear. El primer clic ya va a tiempo pleno ($\approx 0,1$ s los 5 modelos); no hay espera de carga.

- Os acabo de decir que los encoders *cased* se hunden con las mayúsculas. En vez de otra tabla, os lo enseño en vivo.
- Tecleo una frase corta y positiva, pero sutil: “pues no está nada mal”.
- Los cinco modelos la clasifican a la vez —LoRA de 1.7B y 4B, prompting, BETO y XLM-R—. Todos: *positivo*. De acuerdo.
- Ahora pulso el botón de MAYÚSCULAS —el mismo ruido de la robustez— y vuelvo a clasificar la misma frase.
- Y mirad: BETO y XLM-R voltean a *negativo*. Solo por las mayúsculas. Los LoRA no se mueven: siguen en positivo.
- Ahí está el hallazgo, en directo: el encoder *cased* confunde el grito con enfado; el adaptador generativo, no.
- Cierro como empecé: para el español, afinar barato es viable —y aguanta el ruido real—. Gracias; quedo a vuestra disposición para las preguntas.

▷ **Plan B (obligatorio, si la demo falla):**

- Si el túnel o la DGX no responden: arranco el *fallback* en CPU en el portátil (`python demo/app.py --cpu`) —misma demo, LoRA-1.7B, ≈ 1 s por frase—.
- Si tampoco: salto a la *slide de respaldo* con la captura del antes/después (BETO y XLM-R positivo→negativo; LoRA positivo) y lo cuento con la misma frase.

- Regla: nunca me quedo en blanco —la slide de respaldo está en el deck (oculta) y el mensaje es el mismo con o sin directo.

Cifras: Demo: `demo/app.py` (GPU, 5 modelos) / `--cpu` (fallback CPU). Colapso *cased* trazado a `results/robustness_results.csv`. (BETO -0,184; XLM-R -0,110).

Puente al Q&A (10 min) — enlace con el banco 02_banco

Las preguntas más probables ya las he *tocado* en el guion; en el turno de preguntas solo tengo que **rematar con la versión de 10 segundos** del banco. Atajo mental:

- **RoBERTuito (0,758)** → ya pre-emptado en 3b: “fuera de Pareto, otro corpus, sin reentrenar”. (W1)
- **H200 vs T4** → ya pre-emptado en bloque 4: “VRAM pico = cifras del target; solo cambia el tiempo”. (W4)
- **¿Los σ son significancia?** → “magnitud relativa al ruido, no test; y replica en dos modelos”. (W3)
- **Una sola semilla en B** → “ $\pm 0,006$ a datos completos; irrelevante frente a 4–6 puntos”. (W5)
- **¿El factor de parámetros?** → “ $10.000\times$ es de Hu et al.; lo mío es $17\text{--}43\times$ / $9\text{--}24\times$ / $268\times$ ”. (W14)
- **¿Uso de IA?** → “yo orquesté y validé con gates; el diario es la traza”. (CE37/RA5)
- **¿Y la robustez?** → “el único colapso es *cased* ante mayúsculas; LoRA aguanta; lo acabáis de ver en la demo”. (Sprint 6)

Puertas que NO abrir (no las saques tú; si te empujan, recoge y recoloca):

- No compares tiempos de entrenamiento T4 vs H200 (no los medí en T4).
- No digas que Qwen3 es “mejor” que Llama/Gemma (no lo medí): defiende los criterios.
- No defiendas el $10.000\times$ como cifra propia: es de Hu et al.
- No afirmes causalidad dialectal (por qué ES es sensible): el dato es la asimetría, no su causa.
- No presentes la robustez como exhaustiva: son 7 perturbaciones sobre un test; el mayúsculado es el hallazgo, no un *survey*.

Regla de oro: si una pregunta toca un límite, recondúcela a su *encuadre* (“está declarado a propósito porque...”) en vez de improvisar. Respuestas largas en el banco; en la sala, la de 10 segundos.

Herramientas de respaldo (Q&A, *no* en el discurso):

- **Pareto / cruce / transferencia** → abrir `defense/deck_assets/explorer.html` (offline, doble clic): Vista 1 (Pareto, Corte B), Vista 2 (cruce F1-vs-n, Corte A), Vista 3 (transferencia 3×3 , Corte C).
- **Robustez / otras perturbaciones** (tildes, emojis, abreviaturas...) → `figF` (heatmap de robustez, `results/figures/figF_robustness.png`); tabla completa en `results/robustness_results.csv`.
- **¿Otra frase en la demo?** → la demo sigue abierta; pruebo la que propongan (o el modo `--cpu` en el portátil).
- **Trade-off de coste 4D** → `figE` (dispersión 4D, asset de reserva).